

Genie Workbench — Per-Space Column Selection

Feature guide · How it works and how to use it

Overview

Column Selection lets you restrict, **per Genie Space**, which columns of each data source are considered during analysis. It affects all three analysis paths in the Workbench:

- **IQ Scan** — the rule-based scoring engine
- **Create / Fix agent** — the LLM agents that inspect and patch spaces
- **Auto-Optimize (GSO)** — the full benchmark-driven optimization pipeline

It is **opt-in**. When it's off (the default), all columns are used — exactly as before. No redeploy is needed to change a selection; it's stored per space and applied on the next run.

Why use it

A Genie Space's tables often carry far more columns than the questions actually need — ETL metadata, audit fields, unused dimensions. Narrowing analysis to the columns that matter:

- makes IQ Scan and Create-agent profiling focus on relevant fields,
- reduces noise in generated benchmarks and optimization proposals,
- and lets you encode "these are the columns this space cares about" once, per space.

How to use it

Step 1 — Open the panel

In the UI, go to: Space detail → Score tab → Space Configuration → Column Selection.

Step 2 — Turn it on and define the allowlist

Toggle **"Use selected columns only"** and edit the JSON allowlist. The allowlist is keyed by fully-qualified data source (catalog.schema.name — a table, view, or metric view). Example:

```
{
  "data_sources": {
    "main.sales.orders": ["orderid", "amount", "orderdate", "region"],
    "main.sales.salesmetrics": ["region", "ordermonth", "total_revenue"],
    "main.sales.customers": ["*"]
  }
}
```

Rules (allowlist; case-insensitive on both identifier and column name):

- A source listed with a column list → only those columns are used.
- A source listed with ["*"] or an empty list → all columns are used.

- A source not listed → all columns are used.
- Toggle off, or an empty config → all columns everywhere (backward compatible).

For a metric view, list its dimension/measure names (Unity Catalog reports these as columns).

Step 3 — Or: Recommend from usage history

Rather than hand-typing the allowlist, click **Recommend from usage history**. The app inspects Databricks usage history and pre-fills the allowlist with only the columns that have actually been used — which you can then edit before saving.

- **Signal:** reads `system.access.columnlineage`, which records the columns read by each query and attributes them to the Genie Space via `entitymetadata.geniespaceid`. Column-level and authoritative — no SQL-text parsing.
- **Window:** 30 days by default; a column is recommended if it was used at least once in the window.
- **Views and metric views:** their base tables are resolved too (via `information_schema.viewtableusage`, a regex fallback over `viewdefinition`, or the metric-view YAML source), so base-table history counts.
- **New spaces:** if a space has no history of its own yet, it falls back to workspace-wide usage of the same tables so recommendations are still useful.
- Sources with no history are surfaced (empty) so you know to add columns manually or use `["*"]`.

A note summarizes per-source counts, which sources used base-table history, and which had none. Review, then Save.

Step 4 — Save

Save writes the selection to Lakebase. It applies on the next scan / agent run / optimization — no redeploy.

Precedence

The order of resolution is:

1. Per-space UI setting (always wins)
2. Global file fallback (`COLUMNSELECTIONENABLED` + `COLUMNSELECTIONCONFIG` env vars)
3. All columns

The global file is a deployment-wide fallback for spaces that have no per-space setting.

Permissions (for "Recommend from usage history")

The recommendation runs as the app **service principal** — system tables are not readable via user (OBO) auth. The SP needs `SELECT` on `system.access.column_lineage`.

This grant is in the deploy's grant list (`scripts/deploylib/uc.py` → `WATCHSYSTEM_GRANTS`), but granting on `system.*` requires an account admin. Until it's applied, the button degrades gracefully to "usage history unavailable" rather than erroring.

What changed under the hood

- **Storage:** new `genie.spacecolumnselection` table in Lakebase (in-memory fallback when Lakebase isn't configured).
- **Filtering:** new `backend/services/columnselection.py` — *parseallowlist / allowedcolumns / filtercolumns*, with per-call override.
- **Recommendations:** new `backend/services/column_usage.py` — usage-history reads plus view/metric-view base-table resolution.
- **Endpoints:** GET / PUT `/api/spaces/{id}/column-selection`, and POST `/api/spaces/{id}/column-selection/recommend` (in `spaces.py`).
- **IQ Scan:** `scanner.enrichwithucdescriptions` honors the allowlist.
- **Create / Fix agent:** threaded via `handletool/call(columnoverride=)` into *describetable / profilecolumns / assessdataquality / assessreadiness*.
- **Auto-Optimize (GSO):** carried in the run snapshot as *columnselection*, applied at preflight (`config["uccolumns"]`) so all downstream stages inherit the reduced set.
- **UI:** new `ColumnSelectionPanel.tsx` (toggle, JSON editor, Template + Recommend buttons).
- **Tests:** `testcolumnselection.py`, `testcolumnusage.py`.
- **Docs:** `docs/docs/features/column-selection.md`; `AGENTS.md` and env-vars reference updated.

Notes and limits

- "Has usage history" today means used at least once in the window; there is no minimum-count or top-N ranking yet.
- Only read columns (`sourcecolumnname`) count as usage.
- Column Selection scopes what analysis considers — it does not rewrite the deployed Genie Space config or remove columns from the space itself.